

WEEK-3 Practical

Task-1: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process States at different stages of execution

```
pavani@DESKTOP-38OLEIN: ~  
GNU nano 8.7.1 process.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/types.h>  
#include <sys/wait.h>  
#include <stdlib.h>  
  
int main() {  
    pid_t pid;  
    int status;  
  
    printf("Initial Process\n");  
    printf("PID : %d\n", getpid());  
    printf("PPID : %d\n\n", getppid());  
  
    pid = fork();  
  
    if (pid < 0) {  
        perror("fork failed");  
        return 1;  
    }  
  
    if (pid == 0) {  
        /* Child process */  
        printf("CHILD PROCESS\n");  
        printf("PID : %d\n", getpid());  
        printf("PPID : %d\n", getppid());  
        printf("State: Running\n\n");  
  
        printf("Child is doing some work...\n");  
        sleep(5);  
  
        printf("Child work completed.\n");  
        printf("State: Terminated\n");  
  
        exit(0);  
    }  
  
    else {  
        /* Parent process */  
        printf("PARENT PROCESS\n");  
        printf("PID : %d\n", getpid());  
        printf("PPID : %d\n", getppid());  
        printf("Child PID: %d\n", pid);  
        printf("State: Running\n\n");  
  
        printf("Parent is waiting for child...\n");  
        printf("State: Waiting\n");  
    }  
}
```

Output:

```
pavani@DESKTOP-380LEIN: ~  
pavani@DESKTOP-380LEIN:~$ nano process.c  
pavani@DESKTOP-380LEIN:~$ gcc process.c -o process  
pavani@DESKTOP-380LEIN:~$ ./process  
Initial Process  
PID : 615  
PPID : 371  
  
PARENT PROCESS  
PID : 615  
PPID : 371  
Child PID: 616  
State: Running  
  
Parent is waiting for child...  
State: Waiting  
CHILD PROCESS  
PID : 616  
PPID : 615  
State: Running  
  
Child is doing some work...  
Child work completed.  
State: Terminated  
  
Child has terminated.  
Parent State: Running  
Child Exit Status: 0  
pavani@DESKTOP-380LEIN:~$
```

Report: In this task, I developed a C program using the `fork()` system call to create a parent and a child process. The PID and PPID of both processes were displayed using `getpid()` and `getppid()`. The child process performed some work and the parent process waited for the child using `waitpid()`. The different stages of the processes were observed, including running, waiting, and termination. The program executed successfully and showed the relationship between the parent and child processes.

Task-2

```
pavani@DESKTOP-380LEIN: ~  
GNU nano 8.7.1  
#include <stdio.h>  
#include <unistd.h>  
  
int main() {  
    printf("Process started\n");  
    printf("PID  : %d\n", getpid());  
    printf("PPID : %d\n", getppid());  
  
    printf("Process is running...\n");  
  
    sleep(30);  
  
    printf("Process completed.\n");  
  
    return 0;  
}
```

Output:

```
Process completed.  
pavani@DESKTOP-380LEIN:~$ ./state  
Process started  
PID  : 779  
PPID : 371  
Process is running...  
Process completed.  
pavani@DESKTOP-380LEIN:~$
```

Report: In this task, I observed process state transitions in Linux using the ps, top, and /proc monitoring tools. I created a simple C program that remained active for some time using sleep(). Using the ps command, I observed the PID, PPID, and state of the process. The process showed state S, which represents a sleeping or waiting state. The /proc/PID/status file was used to obtain detailed process information, and top was used to monitor the process. From these observations, I understood how a process changes and how its state can be monitored in Linux.